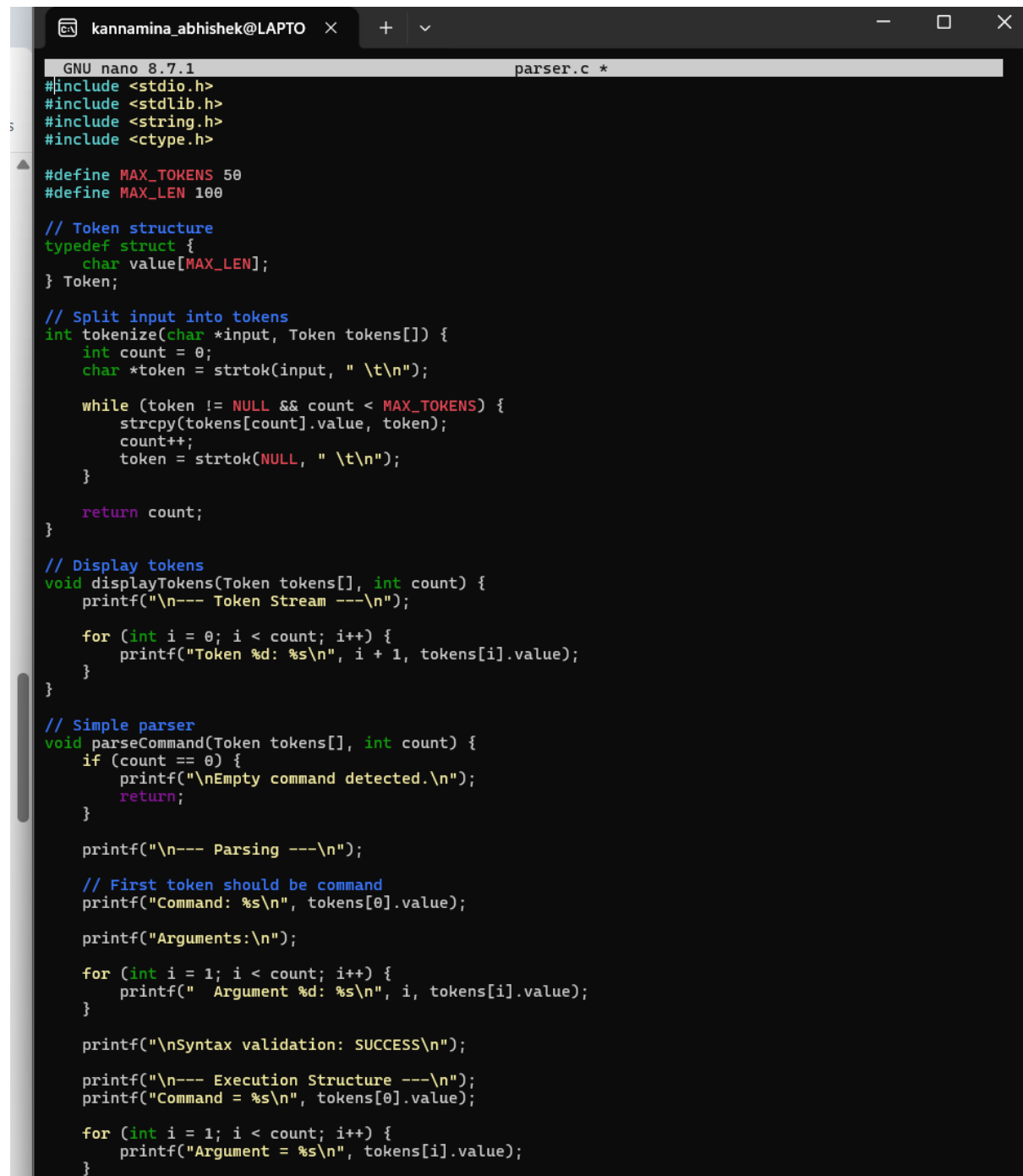


SKILL Q4

1. To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

code:



```
GNU nano 8.7.1 parser.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 50
#define MAX_LEN 100

// Token structure
typedef struct {
    char value[MAX_LEN];
} Token;

// Split input into tokens
int tokenize(char *input, Token tokens[]) {
    int count = 0;
    char *token = strtok(input, " \t\n");

    while (token != NULL && count < MAX_TOKENS) {
        strcpy(tokens[count].value, token);
        count++;
        token = strtok(NULL, " \t\n");
    }

    return count;
}

// Display tokens
void displayTokens(Token tokens[], int count) {
    printf("\n--- Token Stream ---\n");

    for (int i = 0; i < count; i++) {
        printf("Token %d: %s\n", i + 1, tokens[i].value);
    }
}

// Simple parser
void parseCommand(Token tokens[], int count) {
    if (count == 0) {
        printf("\nEmpty command detected.\n");
        return;
    }

    printf("\n--- Parsing ---\n");

    // First token should be command
    printf("Command: %s\n", tokens[0].value);

    printf("Arguments:\n");

    for (int i = 1; i < count; i++) {
        printf("Argument %d: %s\n", i, tokens[i].value);
    }

    printf("\nSyntax validation: SUCCESS\n");

    printf("\n--- Execution Structure ---\n");
    printf("Command = %s\n", tokens[0].value);

    for (int i = 1; i < count; i++) {
        printf("Argument = %s\n", tokens[i].value);
    }
}
```

Output:

```
Command 'cd~' not found, did you mean:
command 'cdo' from deb cdo (2.5.4-1)
command 'cde' from deb cde (0.1+git9-g551e54d-3)
command 'cdi' from deb cdo (2.5.4-1)
command 'cdb' from deb tinycdb (0.81-2build1)
command 'cd5' from deb cd5 (0.1-4build1)
command 'cdw' from deb cdw (0.8.1-3build4)
command 'cdv' from deb erlang-observer (1:27.3.4.6+dfsg-1)
command 'cdp' from deb irpas (0.10-10build1)
Try: sudo apt install <deb name>
kannamina_abhishek@LAPTOP-D9F9IT80:~$ nano parser.c
kannamina_abhishek@LAPTOP-D9F9IT80:~$ gcc parser.c -o parser
kannamina_abhishek@LAPTOP-D9F9IT80:~$ ls
2520090208_OS      client      p2_copy     p4_wait.c   p6_client.cb  sample.txt
2520090208_oss    client.c    p2_copy.c   p5_pipe     p6_server     server
OSLab              client_to_server p3_process  p5_pipe.c   p6_server.c   server.c
OSSP_Project       copy.txt    p3_process.c p6_client    parser         server_to_client
Practical-01       myfifo      p4_wait     p6_client.c  parser.c
kannamina_abhishek@LAPTOP-D9F9IT80:~$ ./parser
Simple Shell Parser
=====

parser> ls -l

Token stream is valid.

--- Token Stream ---
Token 1: ls
Token 2: -l

--- Parsing ---
Command: ls
Arguments:
  Argument 1: -l

Syntax validation: SUCCESS

--- Execution Structure ---
Command = ls
Argument = -l

parser> |
```

```
kannamina_abhishek@LAPTO  ×  +  ▾  -  □  ×
GNU nano 8.7.1  parser.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 50
#define MAX_LEN 100

// Token structure
typedef struct {
    char value[MAX_LEN];
} Token;

// Split input into tokens
int tokenize(char *input, Token tokens[]) {
    int count = 0;
    char *token = strtok(input, " \\t\\n");

    while (token != NULL && count < MAX_TOKENS) {
        strcpy(tokens[count].value, token);
        count++;
        token = strtok(NULL, " \\t\\n");
    }

    return count;
}

// Display tokens
void displayTokens(Token tokens[], int count) {
    printf("\\n--- Token Stream ---\\n");

    for (int i = 0; i < count; i++) {
        printf("Token %d: %s\\n", i + 1, tokens[i].value);
    }
}

// Simple parser
void parseCommand(Token tokens[], int count) {
    if (count == 0) {
        printf("\\nEmpty command detected.\\n");
        return;
    }

    printf("\\n--- Parsing ---\\n");

    // First token should be command
    printf("Command: %s\\n", tokens[0].value);

    printf("Arguments:\\n");

    for (int i = 1; i < count; i++) {
        printf("  Argument %d: %s\\n", i, tokens[i].value);
    }

    printf("\\nSyntax validation: SUCCESS\\n");

    printf("\\n--- Execution Structure ---\\n");
    printf("Command = %s\\n", tokens[0].value);

    for (int i = 1; i < count; i++) {
        printf("Argument = %s\\n", tokens[i].value);
    }
}
```